

RESEARCH QUESTIONNAIRE

Pampanga State University - College of Computing Studies

Python Programming Assessment and Coding Practices

Good day. We are third-year Bachelor of Science in Computer Science students conducting a study on a Web-Based Python Integrated Development Environment (IDE) designed to evaluate advanced programming logic and promote academic integrity among higher-year computer science students. This questionnaire aims to assess your adherence to academic integrity, your current coding practices, and your foundational knowledge of complex Python structures prior to utilizing the system. All responses will be kept strictly confidential and will only be used for academic purposes.

PART 1: Demographics & Background

Name (Optional): _____

Section: _____

How would you rate your current skill level in Python programming?

- ☐ Novice (Can write basic scripts but struggles with advanced logic and object-oriented concepts)
- ☐ Intermediate (Comfortable with data structures, error handling, and standard algorithms)
- ☐ Advanced (Can efficiently use list comprehensions, decorators, and complex architectural patterns)
- ☐ Expert (Can optimize code for performance and build scalable web/software applications)

PART 2: Academic Integrity

Please indicate your level of agreement with the following statements regarding your adherence to academic integrity during programming tasks. (Scale: 1 - Strongly Disagree, 2 - Disagree, 3 - Neutral, 4 - Agree, 5 - Strongly Agree)

1. I strictly write my own logic for take-home programming assignments without copying from classmates.
[1] [2] [3] [4] [5]
2. I utilize generative AI tools to write the entirety of my code rather than formulating the architecture myself.
[1] [2] [3] [4] [5]
3. I submit code snippets copied from external repositories without citing the original source.
[1] [2] [3] [4] [5]
4. I fully understand the ethical implications of claiming another developer's algorithmic logic as my own.
[1] [2] [3] [4] [5]
5. I allow my peers to duplicate my source code during individual laboratory examinations.
[1] [2] [3] [4] [5]
6. I attempt to bypass standard code evaluation restrictions by utilizing secondary devices during coding sessions.
[1] [2] [3] [4] [5]
7. When collaborating, I ensure my individual contribution is logically distinct and not just a direct copy of my partner's work.

[1][2][3][4][5]

8. I rely on copy-pasting functional blocks of code from the internet without analyzing how they integrate into my program.

[1][2][3][4][5]

9. I believe that submitting AI-generated code as my own original work constitutes academic misconduct.

[1][2][3][4][5]

10. I switch tabs frequently during online examinations to search for direct solutions to programming problems.

[1][2][3][4][5]

11. I alter variable names in copied code to make it appear as my original work.

[1][2][3][4][5]

12. I prefer using an IDE that actively tracks behavioral metrics to ensure a fair evaluation of all students.

[1][2][3][4][5]

13. I intentionally seek out pre-solved program files from higher-year students to submit as my own.

[1][2][3][4][5]

14. I am confident in solving complex logical problems independently without relying on unauthorized external assistance.

[1][2][3][4][5]

15. I view plagiarism in software development as a severe violation of professional computing ethics.

[1][2][3][4][5]

16. I utilize unauthorized browser extensions to bypass academic monitoring systems.

[1][2][3][4][5]

17. I ask peers to write complex functions for me when I encounter difficulties, rather than asking for conceptual guidance.

[1][2][3][4][5]

18. I believe that real-time instructor dashboards promote a more honest and transparent coding environment.

[1][2][3][4][5]

19. I report or discourage academic dishonesty when I observe it occurring during programming laboratories.

[1][2][3][4][5]

20. I value the process of independent logical formulation more than merely achieving a passing grade through unauthorized shortcuts.

[1][2][3][4][5]

PART 3: Coding Practices

Please indicate your level of agreement with the following statements regarding your standard software development and coding habits. (Scale: 1 - Strongly Disagree, 2 - Disagree, 3 - Neutral, 4 - Agree, 5 - Strongly Agree)

1. I actively refactor my code to improve its time and space complexity after achieving an initial working solution.
[1][2][3][4][5]
2. I rely entirely on trial-and-error execution rather than logically analyzing the program flow before running the script.
[1][2][3][4][5]
3. I utilize structural feedback, such as Abstract Syntax Tree (AST) analysis, to improve the architecture of my code.
[1][2][3][4][5]
4. I leave redundant variables and unused functions in my final codebase if the program executes without errors.
[1][2][3][4][5]
5. I consistently read and analyze error logs and stack traces before attempting to modify my code.
[1][2][3][4][5]
6. I write inline comments and documentation for complex logical blocks to ensure future readability.
[1][2][3][4][5]
7. I prefer development environments that solely validate output correctness rather than environments that evaluate code structure.
[1][2][3][4][5]
8. I construct custom test cases to verify the robustness of my algorithms against edge-case scenarios.
[1][2][3][4][5]
9. I disregard naming conventions (e.g., PEP 8 for Python) when developing rapid prototypes.
[1][2][3][4][5]
10. I prioritize code efficiency and execution speed when designing advanced looping constructs.
[1][2][3][4][5]
11. I utilize version control principles, even conceptually, to track the iteration of my programming logic.
[1][2][3][4][5]
12. I struggle to maintain focus and frequently switch between non-academic applications while writing complex logic.
[1][2][3][4][5]
13. I design my program's architecture mentally or on paper before typing syntax into the IDE.
[1][2][3][4][5]
14. I find automated hints regarding inefficient structural loops highly beneficial to my long-term skill development.
[1][2][3][4][5]

15. I frequently execute code without understanding how the underlying data structures process the information.
☐ ☐ ☐ ☐ ☐
16. I utilize modular programming by breaking down monolithic scripts into smaller, reusable functions.
☐ ☐ ☐ ☐ ☐
17. I rely heavily on global variables instead of passing arguments through localized function parameters.
☐ ☐ ☐ ☐ ☐
18. I systematically isolate specific code blocks to debug logical errors.
☐ ☐ ☐ ☐ ☐
19. I ignore compiler or interpreter warnings as long as the application does not crash.
☐ ☐ ☐ ☐ ☐
20. I consciously apply Object-Oriented Programming (OOP) principles when the complexity of the problem requires it.
☐ ☐ ☐ ☐ ☐

PART 4: Advanced Python Logic Assessment

Instructions: Please read and analyze each Python code snippet below. Select the correct output or concept. If you do not know the answer, please select "I am not sure."

1. What will be the output of this list comprehension?

Python

- ```
1. nums = [x for x in range(5) if x % 2 == 0]
2. print(nums)
```

☐ [1, 3]

☐ [0, 2, 4]

☐ [2, 4]

☐ I am not sure

2. What is the output of the following code involving variable scope?

Python

- ```
1. x = 10
2. def modify():
3.     global x
4.     x += 5
```

- 5.
6. `modify()`
7. `print(x)`

☐ 10

☐ 15

☐ Error

☐ I am not sure

3. Which of the following best describes the use of the `yield` keyword in this context?

Python

1. `def my_gen():`
2. `yield 1`
3. `yield 2`

☐ It terminates a function and returns a specific value.

☐ It pauses a function to return an intermediate value, creating a generator.

☐ It handles exceptions and prevents the program from crashing.

☐ I am not sure

4. What will be the output of this lambda function?

Python

1. `calc = (lambda x, y: x ** y if x > y else y ** x)`
2. `print(calc(2, 3))`

☐ 6

☐ 8

☐ 9

☐ I am not sure

5. Consider the following Object-Oriented code. What will it print?

Python

```
1. class Animal:
2.     def speak(self): return "Sound"
3.
4. class Dog(Animal):
5.     def speak(self): return "Bark"
6.
7. pet = Dog()
8. print(pet.speak())
```

☐ Sound

☐ Bark

☐ Error

☐ I am not sure

6. What will be the output of this error-handling block?

Python

```
1. try:
2.     result = 10 / 0
3. except ZeroDivisionError:
4.     print("Error")
5. finally:
6.     print("Done")
```

☐ Error

☐ Done

☐ Error followed by Done

☐ I am not sure

7. Which of the following data structures is immutable (cannot be changed after creation) in Python?

Python

```
1. data_a = [1, 2]
2. data_b = {"id": 1}
```

3. `data_c = (1, 2)`

- ☐ List (`data_a`)
- ☐ Dictionary (`data_b`)
- ☐ Tuple (`data_c`)
- ☐ I am not sure

8. What is the primary function of a Python decorator?

Python

1. `@my_decorator`
2. `def function():`
3. `pass`

- ☐ It modifies the behavior of a function or class without permanently changing its source code.
- ☐ It styles the visual output of the printed text in the terminal.
- ☐ It encrypts the source code to prevent unauthorized copying.
- ☐ I am not sure

9. What will the following dictionary comprehension produce?

Python

1. `squares = {x: x*x for x in range(3)}`
2. `print(squares)`

- ☐ `{1: 1, 2: 4, 3: 9}`
- ☐ `{0: 0, 1: 1, 2: 4}`
- ☐ `[0, 1, 4]`
- ☐ I am not sure

10. Based on Abstract Syntax Tree (AST) efficiency, which code block is structurally better for finding unique items?

Python

```
1. # Code A
2. unique = []
3. for i in my_list:
4.     if i not in unique: unique.append(i)
5.
6. # Code B
7. unique = list(set(my_list))
```

☐ Code A

☐ Code B

☐ Both are equally efficient

☐ I am not sure

11. What will the following code utilizing `*args` print?

Python

```
1. def func(*args):
2.     return sum(args)
3.
4. print(func(1, 2, 3, 4))
```

☐ 10

☐ (1, 2, 3, 4)

☐ Error

☐ I am not sure

12. What is the difference between the `is` and `==` operators in this code?

Python

```
1. a = [1, 2]
2. b = [1, 2]
3. print(a == b, a is b)
```

☐ True True

☐ True False

☐ False True

☐ I am not sure

13. What is the output of this `map` function?

Python

```
1. nums = [1, 2, 3]
2. result = list(map(lambda x: x + 1, nums))
3. print(result)
```

☐ [1, 2, 3]

☐ [2, 3, 4]

☐ 6

☐ I am not sure

14. What will happen to the value of `old[0][0]` after this shallow copy operation?

Python

```
1. import copy
2. old = [[1], [2]]
3. new = copy.copy(old)
4. new[0][0] = 9
5. print(old[0][0])
```

☐ 1

☐ 9

☐ Error

☐ I am not sure

15. What will be the output of this string slicing operation?

Python

```
1. text = "PYTHON"
2. print(text[::-1])
```

☐ PYTHON

☐ NOHTYP

☐ P

☐ I am not sure

16. What is the primary purpose of the `__init__` method in a Python class?

Python

1. `class MyClass:`
2. `def __init__(self, val):`
3. `self.val = val`

☐ To delete the object and free up memory.

☐ To initialize the attributes of a newly created object instance.

☐ To execute the main loop of the application.

☐ I am not sure

17. What is the result of utilizing the `zip` function on these two lists?

Python

1. `names = ["A", "B"]`
2. `scores = [10, 20]`
3. `print(dict(zip(names, scores)))`

☐ `{"A": 10, "B": 20}`

☐ `[("A", 10), ("B", 20)]`

☐ Error

☐ I am not sure

18. How does the `with` statement (Context Manager) function in this file operation?

Python

1. `with open("test.txt", "w") as f:`
2. `f.write("Hello")`

- ☐ The file must still be manually closed using `f.close()`.
- ☐ It automatically closes the file even if an exception occurs.
- ☐ It is exclusively used for reading files, not writing.
- ☐ I am not sure

19. What is the output of `enumerate` in this loop?

Python

```
1. items = ["apple", "banana"]
2. for i, v in enumerate(items):
3.     if i == 1: print(v)
```

- ☐ apple
- ☐ banana
- ☐ 1
- ☐ I am not sure

20. What will be the output of this recursive function?

Python

```
1. def fact(n):
2.     if n == 1: return 1
3.     return n * fact(n-1)
4.
5. print(fact(3))
```

- ☐ 3
- ☐ 6
- ☐ 9
- ☐ I am not sure